

NEXUS AI

Master Implementation Plan

Complete Development Execution Plan • Architecture → Core Runtime → AI → RAG → Agents → UI →
Data → Testing → Release

Version: 1.0 **Status:** Implementation Baseline **Target:** Windows Local-first AI Workstation

Purpose: Convert the completed NEXUS AI product, architecture, AI/model, RAG, agent/tool, application, data, testing and release documents into one executable development roadmap. This document defines what is built first, what depends on what, milestone gates, sprint order and the definition of a release-ready MVP.

DESIGN COMPLETE → FOUNDATION → RUNTIME → AI → RAG → AGENTS → APPLICATION → HARDENING → RELEASE

1. Implementation Mission

The objective is to build NEXUS AI as a Windows-installable, local-first AI workstation that can chat with local models, use project context, retrieve private knowledge, execute controlled tools and perform coding-agent workflows without requiring cloud AI for the core MVP.

Implementation Rules

- Build the smallest stable foundation before adding autonomy.
- Keep subsystem boundaries explicit.
- Never implement security as a prompt-only feature.
- Make every long-running operation observable and cancellable.
- Use interfaces around Ollama, vector storage, tools and desktop runtime.
- Do not add multi-agent complexity before single-agent execution is reliable.
- Test each milestone before moving to the next dependent phase.
- Preserve user data across application upgrades.
- Keep the core workflow functional offline.

2. Target MVP

Capability	MVP Definition
Local Chat	Streaming chat using installed Ollama models.
Model Management	Discover installed models, select model and basic routing.
Projects	Create isolated project/workspace.
Knowledge	Import supported documents and perform local RAG.
Citations	Show source references for grounded answers.
Agents	Bounded planner + coding/tool workflow.
Filesystem	Scoped read/write/search operations.
Terminal	Controlled command/test execution.
Memory	Conversation/project memory with explicit lifecycle.
Desktop	Installable Windows application.
Security	Sandbox, permissions, approval and offline boundaries.
Testing	AI/RAG/agent/security/performance/offline release gates.

3. Master Architecture to Build

Desktop Shell → Frontend → Application API → Orchestrator → Model Gateway / RAG / Memory / Agent Runtime / Tool Gateway → Database / Filesystem / Ollama / Local Index.

The implementation must preserve the architectural separation already defined in N07–N16, N17–N45 and N46–N62.

4. Phase Map

Phase	Name	Primary Outcome
P0	Repository & Development Foundation	Buildable repository and local developer environment.
P1	Persistence & Core Services	Database, migrations, configuration and service contracts.
P2	Local AI Runtime	Ollama integration, model registry and inference gateway.
P3	Chat & Context	Streaming chat, conversation persistence and context management.
P4	RAG / Knowledge Engine	Ingestion, chunking, embeddings, search, citations.

Phase	Name	Primary Outcome
P5	Agent & Tool Runtime	Planner, tools, permissions, filesystem and terminal.
P6	Coding Agent	Codebase understanding, edit/test/verify loop.
P7	Application UI	Desktop shell, projects, models, knowledge and agents UI.
P8	Memory & Advanced Intelligence	Project memory, verification, routing and multi-agent foundations.
P9	Integration & Hardening	Full system integration, security and failure recovery.
P10	Testing & Optimization	AI/RAG/security/performance/offline validation.
P11	Packaging & Release	Windows installer, migration, backup and MVP launch.

5. Dependency Graph

P0 → P1 → P2 → P3 → P4 → P5 → P6 → P7 → P8 → P9 → P10 → P11.

Parallel work becomes possible after contracts stabilize: UI primitives can begin during P2; design-system work can proceed during P3/P4; testing infrastructure can begin during P1; packaging preparation can begin during P7.

Hard Dependencies

RAG depends on persistence + AI embedding/model gateway. Agents depend on AI runtime + tool gateway + permissions + state. Coding Agent depends on filesystem + terminal + agent runtime + project workspace. Final UI depends on stable API contracts but can be developed against mocked contracts earlier.

6. P0 — Repository & Development Foundation

Workstream	Implementation Tasks	Exit Gate
Repository	Initialize mono/app structure, README, docs, licenses.	Repository builds.
Runtime	Choose desktop shell, frontend/backend runtime boundaries.	Architecture decision recorded.
Tooling	Package manager, formatting, linting, type checking.	Checks run locally.
Environment	.env/config templates, Python/Node/runtime setup as applicable.	Clean setup succeeds.
Git	Branches, commits, PR rules, ignore files.	Workflow documented.
CI basics	Build + lint + unit test pipeline.	Green baseline.

Deliverable: empty-but-buildable NEXUS application with a verified developer workflow.

7. P1 — Persistence & Core Services

Implement the data foundation before complex AI workflows.

- Database connection and schema migration system.
- Projects, conversations, messages and task tables.
- Configuration service.
- Application health service.
- Structured event/logging service.
- Workspace path abstraction.
- Repository interfaces for models, projects, messages and tasks.
- Backup/export foundation.

Gate

Create project → persist → restart application → data remains intact. Migration can create/update schema safely.

8. P2 — Local AI Runtime

Build the normalized model gateway around Ollama.

Module	Build
Ollama Adapter	Health check, model list, model invocation, streaming.
Model Registry	Store discovered model metadata.
Model Gateway	Normalized request/response contract.
Routing v1	Manual model selection + simple task-based routing.
Fallback	Bounded unavailable/timeout fallback.
Telemetry	Latency, model ID, status and errors.

Gate

NEXUS launches, discovers installed models and produces a streamed local response through the application API without the frontend directly calling Ollama.

9. P3 — Chat & Context

- Conversation repository and message lifecycle.
- Streaming event channel.
- Chat composer and cancellation.
- Context builder with token budgeting.
- Prompt template/version system.
- Model selection UI/API.
- Conversation search/history.
- Error/retry handling.

Gate

User can create a conversation, receive streaming local AI output, stop generation, restart the app and continue the conversation.

10. P4 — RAG / Knowledge Engine

Build deterministic RAG before agentic RAG.

Import → Parse → Normalize → Chunk → Embed → Index → Hybrid Search → Rerank → Context → Answer → Citation.

Sprint	Implementation
RAG-1	Collections, document metadata and ingestion jobs.
RAG-2	PDF/DOCX/TXT/Markdown/code parsing adapters.
RAG-3	Chunking + provenance + document versioning.
RAG-4	Local embedding adapter + vector store.
RAG-5	Metadata filters + lexical search.
RAG-6	Hybrid retrieval + reranking.
RAG-7	Citation mapping + grounding verification.
RAG-8	Golden retrieval dataset + evaluation harness.

Gate

Import a local document → ask a question → retrieve relevant evidence → generate a grounded answer → show correct source reference. Unauthorized collections must never appear.

11. P5 — Agent & Tool Runtime

Implement agent execution as a controlled state machine.

Task → Plan → Validate → Execute Step → Tool Gateway → Observation → State Update → Verify → Complete.

Order	Module
1	Agent/task state machine.
2	Tool registry and schemas.
3	Permission engine.
4	Filesystem read/search.
5	Filesystem write/edit + diff.
6	Terminal/test executor.
7	Approval requests.
8	Cancellation, timeout and retry budgets.
9	Audit/trace events.

Gate

Agent can inspect a scoped workspace and perform approved operations while every tool request is policy-checked. Path traversal, forbidden commands and uncontrolled execution are blocked.

12. P6 — Coding Agent

Build the Codex-like workflow only after P5 is stable.

Step	Implementation
Discover	Workspace tree, project config and repository metadata.
Search	Relevant files/symbols/errors.
Plan	Bounded implementation plan.
Edit	Controlled file changes.
Diff	Show/retain exact changes.
Test	Run configured tests/build/lint.
Repair	Use test observations for bounded fixes.
Review	Check requirements and unrelated changes.
Verify	Confirm actual test results.

Gate

User asks for a project change → NEXUS identifies relevant files → edits them safely → runs tests → reports actual results and changed files.

13. P7 — Application UI

Develop the desktop interface against stable typed API contracts.

UI Sequence	Build
Shell	Window, navigation, runtime status.
Chat	Conversation list, streaming, composer.
Projects	Project creation and workspace view.
Models	Installed models, health and selection.
Knowledge	Collections, import, ingestion and sources.
Agents	Task progress, tools, diffs, approvals.
Settings	Privacy, storage, permissions and runtime.
Diagnostics	Optional advanced operational view.

Gate

A normal user can complete the MVP workflows without developer tools or direct backend interaction.

14. P8 — Memory & Advanced Intelligence

- Conversation summarization/compaction.
- Explicit user/project memory.
- Memory retrieval with scope filtering.
- AI verification/critic system.
- Improved model routing based on benchmark scores.
- Agent recovery/resume.
- Planner/researcher/coder/verifier role definitions.
- Bounded multi-agent collaboration.

Gate

Advanced intelligence improves task quality without reducing safety, explainability or resource stability.

15. P9 — Full Integration & Hardening

Connect every subsystem and remove prototype shortcuts.

Area	Hardening
API	Validation, error contracts, auth/session boundary if needed.
AI	Timeouts, fallback, model unload/resource handling.
RAG	Index consistency, deletion/version behavior.
Agents	Loop detection, cancellation, permission re-check.
Filesystem	Path/symlink controls and write limits.
Terminal	Command policy, timeout, output limits.
Data	Transactions, migrations, backup checkpoints.
UI	Offline/error/recovery states.

16. P10 — Testing & Optimization

Execute the N63–N69 test program.

- Unit/integration/API regression.
- AI benchmark baseline and regression.
- RAG golden dataset evaluation.
- Agent safety abuse cases.
- Security testing.
- Performance on target hardware.
- Offline network-disabled acceptance.
- Data backup/recovery validation.
- Long-running stability testing.

Gate

No unresolved P0 defects. P1 defects require explicit release approval. AI/RAG/security/offline gates pass.

17. P11 — Packaging & Release

- Build production frontend/backend/runtime.
- Create Windows installer.
- Validate fresh install.
- Validate upgrade/migration.
- Validate uninstall/data-retention behavior.
- Package release metadata and notes.
- Run MVP release checklist.
- Create backup/recovery artifact.
- Perform final smoke test.
- Tag release.

Gate

MVP Release Checklist N72 is fully passed and release candidate survives final installation + offline + smoke validation.

18. Suggested Sprint Structure

Use one- to two-week sprints depending on available development time. A sprint should end with a working increment, not only documentation.

Sprint	Primary Goal
S01	Repository + development environment
S02	Database + configuration + health
S03	Ollama adapter + model registry
S04	Model gateway + streaming
S05	Chat persistence + context
S06	Project/workspace foundation
S07	Document ingestion
S08	Chunking + embeddings + indexing
S09	Hybrid retrieval + citations
S10	RAG evaluation + hardening
S11	Agent runtime + tool registry
S12	Filesystem + permissions
S13	Terminal + testing tools
S14	Coding agent
S15	Desktop UI integration
S16	Knowledge + model + agent UI
S17	Memory + verification
S18	Integration + security hardening
S19	Performance + offline + recovery
S20	Packaging + release candidate

19. Backlog Priority System

Priority	Meaning
P0	Required for MVP / blocks release.

Priority	Meaning
P1	Important capability; should ship if stable.
P2	Enhancement; can follow MVP.
P3	Future/experimental.

Implementation Rule

Never pull P2/P3 work into a sprint when a P0/P1 dependency is incomplete.

20. Definition of Ready

- Requirement is clear.
- Dependencies identified.
- Acceptance criteria written.
- Required API/data contract known.
- Security implications identified.
- Test approach identified.
- Task is small enough to complete in one sprint.

21. Definition of Done

- Implementation complete.
- Code formatted/linted/type-checked where applicable.
- Unit/integration tests added.
- Relevant AI/RAG/security tests added where applicable.
- Error and empty states handled.
- Logs/telemetry added for important operations.
- Documentation updated.
- Code reviewed.
- Acceptance criteria passed.
- No known P0/P1 regression.

22. Developer Workflow

ISSUE → PLAN → BRANCH → IMPLEMENT → LOCAL TEST → COMMIT → REVIEW → MERGE → CI → INTEGRATION TEST → UPDATE ROADMAP.

Commit messages should describe the actual change. Keep branches short-lived. Avoid large mixed-purpose commits.

23. Daily Development Loop

1. Pull latest changes → 2. check active sprint → 3. select one task → 4. inspect existing architecture → 5. implement smallest change → 6. test → 7. update task → 8. commit → 9. document blockers.

AI-Assisted Development

AI coding tools may be used, but generated code must follow the NEXUS architecture and pass the same tests as manually written code. Do not allow an AI coding tool to invent undocumented architecture when a contract already exists.

24. Milestone Gates

Milestone	Required Proof
M1 Foundation	Clean setup + DB + tests.
M2 Local AI	Ollama discovery + streamed inference.
M3 Chat	Persistent local chat.
M4 Knowledge	Cited local RAG.
M5 Agent	Safe tool execution.
M6 Coding	Edit + test + verify loop.
M7 Product	Integrated desktop UI.
M8 Hardened	Security + performance + offline.
M9 Release	Installer + recovery + final checklist.

25. Risk Register — Implementation

Risk	Mitigation
Scope explosion	Freeze MVP and defer advanced features.
AI quality instability	Benchmark models/prompts and keep baselines.
Agent unsafe behavior	External policy engine + sandbox + negative tests.
Local resource limits	Resource-aware routing and concurrency.
RAG poor retrieval	Golden dataset + hybrid search + reranking.
Database migration failure	Versioned migrations + backups.
Desktop packaging issues	Start installer validation before final release.
AI-generated code defects	Tests, code review and architecture gates.
Integration complexity	Stable typed contracts and milestone gates.

26. What NOT to Build First

- Do not start with unrestricted autonomous multi-agent behavior.
- Do not build a cloud synchronization system for the MVP.
- Do not support every document format initially.
- Do not implement every possible tool.
- Do not optimize models before baseline benchmarks exist.
- Do not build advanced memory before conversation/project persistence works.
- Do not polish every UI screen before core workflows function.
- Do not allow arbitrary terminal/system access.

27. MVP Critical Path

Repository → Database → Ollama → Model Gateway → Chat → Projects → RAG → Tool Gateway → Permissions → Filesystem → Terminal → Coding Agent → UI Integration → Security → Performance → Offline → Installer → Release.

This is the shortest path to a meaningful NEXUS AI product while keeping each layer testable.

28. Future Expansion After MVP

Area	Future Capability
Models	Automatic benchmark-driven routing, model packs, specialized models.
RAG	Graph RAG, multimodal documents, advanced agentic research.
Agents	More specialists, scheduling, long-running workflows.

Area	Future Capability
Tools	Browser, GitHub, package manager and controlled integrations.
Robotics/IoT	Optional local device/robot control through explicit connectors.
Collaboration	Shared projects and optional secure synchronization.

29. Final Implementation Command

The development team should treat this document as the execution map and the N01–N72 document set as the specification source. If implementation conflicts with a specification, stop and resolve the architecture decision rather than silently changing the product.

Every completed phase must leave the repository in a runnable state. A phase is not complete because code exists; it is complete when its acceptance gate passes.

30. Master Completion Criteria

- Application installs on Windows.
- Local Ollama models are detected and usable.
- Chat works fully offline.
- Projects and conversations persist.
- Local documents can be indexed and searched.
- Answers can be grounded and cited.
- Agents can safely inspect/edit project files.
- Terminal/test execution is permission-controlled.
- Coding agent can implement and verify a bounded change.
- Memory and task state persist appropriately.
- Security, performance and offline tests pass.
- Backup and recovery are validated.
- MVP installer and release checklist pass.

31. Final Project Roadmap

NEXUS AI is now ready to move from documentation into implementation. The correct development strategy is incremental: build the foundation, prove local inference, establish knowledge, establish safe tools, create the coding agent, integrate the desktop product, then harden and release.

The first actual coding task is **P0-S01: Repository & Development Foundation**. No AI feature should be implemented before the repository, environment, architecture boundaries, configuration and baseline test workflow are established.

Document: NEXUS AI Master Implementation Plan • **Version:** 1.0 • **Status:** Development Ready • **Source:** Completed N01–N72 architecture/specification set